DARKSKY NEXUS

w033

Troubleshooting Guide

Diagnostics, fixes, and known issues

Quick Diagnostics

Installed the .dmg or Windows installer? Skip straight to item 3 — Python and its packages are bundled, there's nothing to check for items 1-2. Those two only apply if you're running NEXUS from Python source.

Before checking specific issues, confirm:

1. (source only) Python version is 3.9 or later: `python3 --version`
2. (source only) Required packages installed: `pip show numpy websockets`
3. NEXUS is running (check the Dock/Task Manager for the app, or the terminal shows no crash if running from source)
4. Browser is pointing at `http://127.0.0.1:8888`
5. No other application is using port 8888 or 8889

Connection Issues

FT8 INTERNAL band quick-tune / Hop mode: no decodes, main waterfall doesn't change (fixed in w032)

Symptom: Clicking a band chip (or letting Band Hopping switch bands automatically) in the FT8 INTERNAL panel updates the VFO frequency digits, but the main spectrum/waterfall stays on the old band, the tuning cursor appears at the far left edge of the display, and the FT8 mini-spectrum/waterfall shows little more than noise — Decodes stays at 0 indefinitely.

Cause: the band quick-tune and Band Hopping band-switch code sent 5 raw low-level SDRConnect commands directly, bypassing NEXUS's own tune-coordination logic (which re-asserts the demodulator ~350ms after a genuine LO move — SDRConnect/nRSP-ST otherwise silently drops the retune). The result: the radio kept listening to the previous band while the UI looked like it had switched.

Fix: band quick-tune and Band Hopping now use the same tuning call every other part of NEXUS uses. Hard-reload the browser (Cmd+Shift+R / Ctrl+Shift+R) to pick up this frontend-only fix. If you still see 0 decodes after switching bands, confirm the LO readout (below the VFO) actually changed to match — if it didn't, that points at an SDRConnect-side issue rather than this bug.

"OFFLINE" shown in top bar — not connecting to SDRConnect

Cause: SDRConnect is not running, or is on a different host/port.

Fix:

- Start SDRConnect first, then start NEXUS
- If SDRConnect is on another machine, NEXUS needs the `--sdr <IP>` flag. Running from source: `python3 w033_NEXUS.py --sdr 192.168.1.xx`. Running the installed app: launch its executable from Terminal/Command Prompt with the same flag instead of using the Applications/Start Menu icon — e.g. on macOS, `"/Applications/DARKSKY NEXUS w033 macOS.app/Contents/MacOS/DARKSKY NEXUS w033 macOS" --sdr 192.168.1.xx`.

- Confirm SDRConnect WebSocket is on port 5454 (default)
- Check your firewall is not blocking port 5454

SDRConnect opens a Finder window asking you to pick an IQ file

Cause: This is SDRConnect's own behaviour, not NEXUS. It tends to happen when SDRConnect is launched first and interacted with while it's still settling into its device-enumeration startup, causing it to fall back to its file-based IQ source picker instead of finding the SDRplay device.

Fix:

- Cancel the file picker, select your SDRplay device in SDRConnect's own device list, and confirm it is actively streaming (waterfall moving in SDRConnect itself)
- **Recommended launch order: start NEXUS first, then SDRConnect.** NEXUS will idle (OFFLINE status) waiting for SDRConnect's WebSocket, which gives SDRConnect a clean, undisturbed startup and reliably avoids this prompt
- If this keeps happening, check SDRConnect's settings for a remembered "last input source" set to a file, and change it back to your SDRplay device

Browser shows blank page or "Cannot connect"

Cause: NEXUS HTTP server is not running or is on a different port.

Fix:

- If running from source, check the terminal — look for `HTTP server on http://127.0.0.1:8888`
- If port 8888 is in use by another app: quit whatever else is using it (most likely cause), or — if running from source — edit `HTTP_PORT` near the top of the .py and restart. The installed app doesn't expose a settings file for this; freeing the port is the practical fix.
- Try opening `http://127.0.0.1:8888` manually in a new tab

"Version mismatch" toast in browser

Cause: Browser has cached an old version of the HTML file.

Fix: Press **Cmd+Shift+R** (macOS) or **Ctrl+Shift+R** (Windows/Linux) to hard-reload.

Waterfall is visible but VFO shows wrong frequency

Cause: SDRConnect has changed the centre frequency independently.

Fix: Click on the waterfall to re-sync, or type the frequency into the VFO display.

ANTENNA control doesn't appear in the status bar

Cause A: Your device only has one antenna input (e.g. RSP1A, RSP1B) — SDRConnect doesn't report a `valid_antennas` list for it, so NEXUS correctly hides the control.

Cause B: NEXUS hasn't received device properties from SDRConnect yet.

- Wait a few seconds after connecting, or check the connection status shows SDRConnect (green).

- Hard-reload the browser (Cmd+Shift+R / Ctrl+Shift+R) if you recently updated NEXUS.

Cause C: SDRConnect itself doesn't expose antenna switching for your firmware/driver version.

- Confirm you can switch antennas from within SDRConnect's own UI. If SDRConnect can't do it, NEXUS can't either — NEXUS only relays what SDRConnect reports.

Selecting an antenna in NEXUS has no effect

Cause: The `active_antenna` property write may not have been accepted by SDRConnect (e.g. device busy, streaming actively).

Fix:

- Check SDRConnect's own antenna selector — it should reflect the change if NEXUS's request succeeded.
- Try stopping and restarting the stream in SDRConnect, then reselect the antenna in NEXUS.

Waterfall & Spectrum Issues

IQ Lite / Compact mode: no audio, no IQ-dependent decoder output (this is expected behaviour, not a bug)

Symptom: Spectrum and waterfall display normally, but there is no audio and decoders that need raw IQ (WEFAX, ACARS, POCSAG, AIS) never produce output, while SDRConnect's stream mode shows **IQ Lite** or **Compact**.

Cause: Confirmed as of w0.2.3, via SDRplay's own official C# reference client (`SDRconnectWebSocketAPI.RawIqWriter`), direct packet capture, and direct confirmation from SDRplay support: **IQ Lite and Compact modes never deliver binary type-2 IQ frames on the nRSP-ST.** Both modes exist only for demodulated/decoder-rate audio streaming at low bandwidth, not raw IQ. This was previously suspected to be the same SDRConnect WebSocket defect tracked under SDRplay support ticket #726970 — it is not the same issue. It is expected behaviour for these stream modes, by SDRplay's own design, not a defect to be fixed.

Fix: Switch SDRConnect's stream mode to **Full IQ**. As of w0.2.3 this is NEXUS's default auto-selected mode for SDRplay nRSP-ST/RSPdx devices, so a fresh connect should already be in Full IQ — if you're still seeing this, check the stream-mode indicator (top bar / status bar) and switch manually in SDRConnect if it's showing IQ Lite or Compact.

SDRplay support ticket #726970 — status update for w0.2.3

Earlier docs described a confirmed SDRConnect bug where no binary IQ (type 0x0002) frames ever arrived over the WebSocket, "reproduced in both IQ Lite and Full IQ modes." Re-testing for w0.2.3 found:

- The **IQ Lite** half of that finding was a misdiagnosis — IQ Lite never carries IQ frames by design (see above), so its lack of IQ frames was never actually evidence of the SDRConnect bug.

- The **Full IQ** half has now been retested directly against SDRplay's own RawIqWriter reference client and **sustained 2 MSps Full IQ cleanly over Wi-Fi for 30+ seconds with zero dropped or irregular frames** — Full IQ works.

> **Resolved for Full IQ.** Full IQ is now treated as working and is NEXUS's default stream mode for SDRplay devices. If you still encounter missing IQ frames specifically in Full IQ mode, that would be a new/different symptom from what ticket #726970 originally described — note the exact stream mode, NEXUS version, and SDRConnect version if you report it.

Waterfall is black / not updating

Cause A: SDRConnect is not streaming (device not connected or not playing).

- Check SDRConnect shows a live spectrum. Press Play in SDRConnect.

Cause B: Browser tab is in background and has throttled.

- Bring the NEXUS browser tab to the foreground.

Cause C: NEXUS just started and hasn't received the first frame yet.

- Wait 2–3 seconds.

Waterfall colours look wrong

Fix: Click a different colour palette swatch in the toolbar. The colour is applied in real time.

Waterfall height takes up too much / too little space

Fix: Drag the resize handle (the thin bar between waterfall and tab panels) up or down. The position is saved for future sessions.

Text/UI is too small to read comfortably (fixed in w0.2.6)

Cause: Earlier builds had no in-app way to scale the interface — the only workaround was the browser's own zoom (Ctrl/Cmd +/-), which several public beta testers reported was still too small even on a 27" 1920×1080 monitor.

Fix: Use the **A− / 100% / A+** control in the top bar to scale the entire app from 80% up to 160%. Your chosen size is remembered across launches (`localStorage`). This scales every panel, button, and label together — not just the waterfall — and is the recommended approach over browser zoom now.

Tuned signal always sits at the left or right edge of the waterfall, never centred (fixed in w0.2.6)

Cause: At the default 1× zoom level, the visible span was always centred on the SDR's hardware centre frequency (`liveCenter`), not on the tuned VFO. If you tuned away from hardware-centre, the signal would render off to one side with no way to recentre the *view* without also zooming in (zooming in already auto-recentred on the VFO as a side effect — there just wasn't an equivalent for the default 1× view).

Fix: Click the new **CTR** button next to the ZOOM controls in the toolbar. It recentres the visible waterfall/spectrum span on your currently tuned VFO frequency, at any zoom level including 1×. This is purely a display change — it does not retune the radio or move the hardware centre frequency. Zooming back out to 1× (or using zoom RST) releases the centring and returns to the default hardware-centred view.

Band plan strip, waterfall, and spectrum all freeze on the old frequency after changing bands (fixed 2026-07-14)

Symptom: After using **CTR** (or zooming in) to recenter the display on a signal, changing to a different band — via the band plan strip, the Bands panel, or any quick-tune button — retunes the radio correctly (the VFO digit readout is correct) but the band plan strip, frequency axis, spectrum trace, and waterfall all stay frozen on the old frequency range.

Cause: CTR sets a display-only recentering point (`zoomCenter`) that took priority over the radio's real hardware centre frequency when working out what span to draw — and nothing cleared it again on a later band change, so the display kept anchoring itself to wherever CTR was last used, even many MHz away from the new tuning.

Fix: Fixed — any deliberate band change now clears that recentering point automatically, so the whole display (band plan, axis, spectrum, waterfall) always follows the real new frequency. CTR still works exactly as before for recentering on a signal at your current tuning.

Spectrum trace not visible

Fix: Click the **SPECTRUM** button in the toolbar to toggle it on.

Spectrum looks erratic / jumps around (Full IQ mode) — fixed in w0.2.3

Cause: A prior build briefly had NEXUS computing and broadcasting its own FFT from raw Full IQ samples, in addition to displaying SDRConnect's own native spectrum — two independent spectra (different bin counts and scaling) interleaving on the same canvas looked like erratic jumping. This has been disabled in w0.2.3; only SDRConnect's native spectrum is shown for Full IQ, which is smooth and correct.

Fix: If you still see this, hard-reload the browser tab (Cmd+Shift+R / Ctrl+Shift+R) to make sure you're running the current HTML, not a cached older version.

Frequency / Tuning Issues

Clicking the waterfall doesn't change frequency

Cause: Click landed outside the waterfall canvas, or the BW overlay edge was clicked.

Fix: Click in the centre of the waterfall, away from edges.

Dragging the BW slider causes the VFO tuning bar to jump (fixed July 2026)

Symptom: After resizing the bandwidth overlay by dragging either the left or right BW edge, the VFO tune-line jumps to a different position on the waterfall — often to wherever the mouse happened to be when the drag ended.

Cause: A BW edge drag ends with a mouseup event. The mousedown on the BW edge called stopPropagation(), which suppresses the mousedown from reaching the waterfall container — but stopPropagation on mousedown does NOT prevent the browser from firing a separate click event after mouseup. If the mouseup occurred anywhere inside the waterfall area, the browser fired a click on the waterfall container and the click-to-tune handler ran, moving the VFO to the cursor's final position. The DS._anyDragging flag was already set during the drag but the click handlers never checked it.

Fix: Added an early-return guard to both the waterfall and spectrum click handlers: if DS._anyDragging is true (still set from the BW drag, cleared 50 ms after mouseup), the ghost click is swallowed and the VFO stays put. This is a frontend-only fix. Hard-reload the browser (Cmd+Shift+R / Ctrl+Shift+R) to pick it up — no Python restart needed.

VFO and LO seem reversed / frequency jumps unexpectedly

Cause: SDRConnect auto-recentred after a large tune. This is normal behaviour — NEXUS will follow.

Cannot tune outside a certain range

Cause: SDRConnect has set a span limit. Increase the span in SDRConnect, or use NEXUS band buttons to jump.

Decoder Issues

Can't find the collapsible category headers in the DECODERS dropdown anymore (changed in w033, not a bug)

This is intentional. w033 replaces the old dropdown-with-collapsible-headers with a floating panel — category tabs (COMPACT / FULL IQ / EXTERNAL) across the top, decoder buttons in a grid below — matching the BANDS control's interaction. See the User Manual, Section 6, for the current layout. w032 still uses the original dropdown-menu style; this redesign is w033-only.

No decoded text from CW / RTTY / PSK31 / NAVTEX (NEXUS engine)

Cause 0 (check this first): Engine toggle is actually set to FLDIGI, not NEXUS.

- The **NEXUS | FLDIGI** toggle in the CW/RTTY header is saved per-decoder and restored automatically on every launch (see User Manual section 6.1). If it was switched to FLDIGI in an earlier session — even briefly, even by accident — it stays on FLDIGI until changed back, with no separate reminder.
- When this happens, pressing **▶ Start** silently routes through fldigi instead: no NEXUS waterfall, no NEXUS decoded-text panel, and no error — because nothing is wrong with the NEXUS decoder, it was simply never asked to run.

- **Fix:** click **NEXUS** in the engine toggle, then ▶ **Start** again. NEXUS now also shows a toast — "CW engine is set to fldigi — switch to NEXUS for the built-in waterfall/Skimmer" — the moment Start is pressed while on FLDIGI.

Cause A: No audio reaching the decoder.

- SDRConnect must be outputting audio to a virtual audio device (BlackHole/VB-Audio)
- Check SDRConnect audio output settings

Cause B: Wrong mode or bandwidth.

- CW: use CW mode, BW 500 Hz
- RTTY: use USB mode, BW 1000 Hz
- PSK31: use USB mode, BW 500 Hz

Cause C: Decoder not started.

- Click ▶ **Start** in the decoder header.

Cause D: Wrong tone frequency (RTTY).

- Drag the M/S markers in the tone scope to the signal tones, or enable Auto-detect.

CW mini-waterfall goes blank the moment Start Decode is pressed, resumes on Stop (Full IQ, fixed July 2026)

Symptom: The CW mini-waterfall scrolls normally before pressing ▶ **Start**. The instant Start is pressed, the waterfall goes completely dark — no new rows. Press ■ **Stop** and the waterfall immediately resumes. The keying scope and decoded-text panel continue working throughout.

Cause (Full IQ only): After the 5× decimation step that brings the full hardware sample rate down to a ~50 kHz audio passband, the passband is only ±25 kHz around the hardware LO. When the CW decoder is active, the audio FFT code computed the carrier bin position from the raw VFO offset from the hardware LO (vfo_hz − center_hz). On HF, the LO can be tens of kHz from the VFO — a 30 kHz offset pushes the carrier bin to position 4507 in a 4096-bin FFT, outside the array. The resulting empty slice suppressed the audio_fft broadcast entirely, blanking the mini-waterfall. Before Start was pressed, audio_fft_center_hz() returned 1500 Hz (a safe audio tone), keeping the bin inside the array — which is why the waterfall was fine before Start and blank after.

Fix: When the CW NEXUS decoder is active in Full IQ mode, the audio FFT block now applies a complex frequency shift to a separate buffer before computing the FFT, translating the VFO signal to DC. The carrier is reported at 0 Hz. This is a backend-only fix (Python). Restart the Python server and hard-reload the browser (Cmd+Shift+R / Ctrl+Shift+R) to pick it up. Also confirm the NEXUS engine is selected (not FLDIGI) — see Cause 0 above.

CW mini-waterfall freezes permanently after pressing Start

Symptom: The small CW waterfall display goes static right after Start is pressed — no new lines scroll in — while everything else (decoded text, ACTIVITY scope) keeps working.

Cause: Not fully reproduced, but the most likely failure mode was a one-time rendering exception (e.g. during a colour-map lookup) leaving shared waterfall render state stuck, so every subsequent frame silently failed the same way with no error shown.

Fix (hardened in w0.2.6): Rendering errors in the CW mini-waterfall are now caught and logged to the browser console instead of failing silently, and a failed colour-map rebuild falls back to a built-in per-pixel colour function instead of leaving the waterfall stuck. If you still see a permanent freeze after updating, open the browser console (Cmd+Option+J / Ctrl+Shift+J) and check for a logged error — that detail is what's needed to chase down any remaining cause.

CW Skimmer shows "ACTIVE" with a visible signal but Skimmer Channels stays at 0 (fixed in w031)

Symptom: The CW Skimmer clearly shows activity on its own mini-waterfall/spectrum — a strong, visually narrow carrier is on screen — but the Skimmer Channels counter never leaves 0, and no channel is ever opened for it.

Cause: The Skimmer's candidate-peak detector rejects any peak wider than a fixed 2.0 kHz threshold, on the assumption that a real CW carrier is narrow and wide peaks are noise/interference. That threshold never accounted for the waterfall's actual Hz-per-bin resolution, which depends on the current zoom/span. At typical zoom levels, a single FFT bin's width alone can exceed 2.0 kHz — meaning even a genuinely narrow, strong CW signal always measured as "too wide" by the detector's own bin-width arithmetic and was rejected before ever being considered a candidate.

Fix: The width filter's threshold now scales with the live Hz-per-bin value ($\max(2.0\ \text{kHz},\ \text{hzPerBin} \times 2.5)$), so it stays meaningful at any zoom level instead of being a fixed value that only worked by coincidence at some specific span. Hard-reload the browser (Cmd+Shift+R / Ctrl+Shift+R) to pick up this frontend-only fix.

Skimmer channels appear in the list but their decoded text stays permanently empty (fixed in w032)

Symptom: Channels show up in the Skimmer Channels list with a frequency, sometimes persisting for many seconds, but the callsign/text columns never populate — as if every candidate were a dead signal.

Cause: Candidate detection ran against the mini-waterfall's own display bins — a per-frame, self-normalising, log-compressed array with no fixed relationship to real signal-to-noise ratio. Measured live, most promoted candidates sat at only 0–1.5 dB actual SNR against the decoder's 8 dB decode threshold, so they were real peaks in the display but never strong enough, in the IQ domain, for the decoder to extract anything from.

Fix: On IQ-Lite/nRSP-ST connections, candidate detection now runs server-side against the same real-dB wideband FFT array the display itself is built from, before it's compressed for the waterfall — so a candidate's reported SNR is the same measurement the decoder will judge it against, and low-SNR candidates are filtered out before ever reaching the channel list. Full IQ and Compact-mode streams still use the previous display-bin scan as a fallback for now.

CW mini-waterfall never starts in Compact mode even though the decoder is running (fixed in w031)

Symptom: CW decode starts normally (keying scope moves, decoded text appears, Skimmer channels populate) but the small CW mini-waterfall panel never draws anything — specifically on Compact-mode streams.

Cause: The server's `audio_fft` broadcast — the mini-waterfall's data source — was computed for IQ-Lite and Full IQ streams only. Compact mode had no equivalent computation at all, so the mini-waterfall had no data to draw on that stream mode regardless of how long decode ran.

Fix: Added the missing Compact-mode `audio_fft` computation (a one-sided FFT over the already-demodulated audio, centred on the CW decoder's live tone reading), so the mini-waterfall now gets a broadcast in every stream mode. Restart the Python server and hard-reload the browser (Cmd+Shift+R / Ctrl+Shift+R) to pick this up.

CW mini-waterfall/spectrum stays a fixed ±3000 Hz scope, independent of the BW setting

This is expected behaviour, not a bug, on IQ-Lite and Full IQ streams. The CW mini-waterfall and the spectrum trace above it are both fed by a server-side FFT computed from the genuine, unfiltered complex IQ signal — the same signal the CW decoder mixes down for its own narrow decode filter, captured *before* that filter is applied. Narrowing or widening the CW decoder's BW setting changes what the decoder listens to and decodes, but has no effect on what the mini-waterfall/spectrum shows: a fixed ±3000 Hz close-up view centred on the VFO, the same width FT8 INTERNAL's own waterfall uses.

On Compact mode, the mini-waterfall is instead built from the already-demodulated, BW-filtered audio (there's no unfiltered IQ available once SDRConnect has demodulated to audio), so on that stream mode specifically, narrowing the BW setting genuinely does narrow what's visible. This is a hardware/stream-mode limitation, not something NEXUS can work around — switch to IQ-Lite or Full IQ mode for the wideband, BW-independent view.

Keying scope jumps sideways as the SNR/tone/speed numbers change (fixed in w031)

Symptom: The CW keying scope visibly shifts left/right each time the TONE, SPEED, or SNR readout above it updates to a value with a different number of digits (e.g. `9.2 dB` → `12.4 dB`).

Cause: The stat boxes had no fixed width, so the box itself resized to fit whatever text was in it, nudging the whole row (and the scope below it) sideways on every update.

Fix: The stat boxes now have a fixed minimum width and use tabular (fixed-width) digit rendering, so the layout no longer shifts regardless of the values shown. Hard-reload the browser to pick up this frontend-only fix.

CW decoded-text panel stuck on "Click Start" even though Skimmer channels are decoding (fixed in w031)

Symptom: The Skimmer Channels list is visibly showing live decoded callsigns and text, but the single-channel decoded-text panel next to it still shows the idle "Click ▶ Start above to begin decoding" placeholder, as if nothing were running.

Cause: The Skimmer pool and the single-channel dial-frequency decoder write to two separate panels (kept apart deliberately so their text doesn't interleave into a garbled mix). When only the Skimmer pool has produced decodes, the single-channel panel has genuinely received no text of its own, so its idle placeholder never had a reason to clear.

Fix: The placeholder's wording now updates (without changing which panel it lives in) to point at the Skimmer Channels list once Skimmer decodes start arriving, as long as the single-channel decoder itself still hasn't produced anything. As of this build, the single-channel panel is also no longer permanently on screen — see "CW tab layout" in the User Manual for the new SKIMMER/SINGLE toggle.

BW dropdown renders far wider than its short preset list (fixed in w031)

Symptom: Clicking the **BW ▾** button opens a dropdown that stretches almost the full width of the window, even though it only lists 4-8 short bandwidth presets — visibly out of proportion next to the properly-sized **Bands ▾** dropdown.

Cause: A leftover CSS rule from an earlier version of this element still declared `right: 0`. The current implementation only ever sets the dropdown's `left`/`top` position at runtime and never touches `right` — so with the panel's `position: fixed` plus a computed `left` and that leftover `right: 0` both in effect, the browser stretched the box to fill the gap between them instead of sizing it to its content.

Fix: The dropdown's inline style now explicitly sets `right: auto`, overriding the stale rule, and the preset grid was tightened from 3 to 2 columns to match the visual weight of the Bands dropdown. Hard-reload the browser to pick up this frontend-only fix.

FT8 INTERNAL monitor audio sounds distorted / too loud (fixed in w031)

Symptom: Starting FT8 INTERNAL decode produces audibly distorted/crackly playback through the speakers, sometimes loud enough to require turning the computer's own volume down.

Cause: FT8 INTERNAL compensates for SDRConnect's own volume response being non-linear (disproportionately quiet) below about 42%, by boosting the audio it plays back locally. That boost had no upper limit — at low SDRConnect volume settings it could reach 20x or more — and every sample was then hard-clamped to full scale, which produces exactly the kind of crackling distortion described.

Fix: The boost is now capped at 1.6x, and the hard clamp was replaced with a soft-knee curve (`tanh`) that rounds off loud peaks smoothly instead of flat-topping them. A new **MON VOL** slider was also added to the FT8 INTERNAL toolbar (default 50%) so the local monitor volume can be turned down further to taste — it only affects what you hear through your speakers, not what the decoder itself analyses, so turning it down never hurts decode quality. Hard-reload the browser to pick up this frontend-only fix.

Multimon-ng decode quality worse than expected on Compact / IQ-Lite paths (fixed in w031)

Symptom: POCSAG/FLEX/DTMF/etc. decode rate via Multimon-ng was noticeably worse on the Compact-mode and IQ-Lite audio paths than the same signal on the Full IQ path.

Cause: `MultimonDecoder.process()` was written to expect genuine raw IQ and does its own FM discriminator internally as the first processing step. On the Compact-mode and IQ-Lite code paths, however, the data being handed to it was already-demodulated audio (SDRConnect had done the FM discrimination already) — running a second, redundant discriminator pass on already-demodulated audio produces nonsense, since there's no IQ phase rotation left to discriminate.

Fix: The decoder is now split into `process()` (for genuine raw IQ — Full IQ path, still runs the discriminator) and a new `process_audio()` (for already-demodulated audio — Compact/IQ-Lite/RTL paths, skips the discriminator and starts from the filtering stage). Each call site now uses whichever is correct for its data. This is a backend-only fix; restart the Python server to pick it up.

No decoded text from CW / RTTY / PSK31 / NAVTEX (FLDIGI engine)

Cause A: fldigi is not running.

- Start fldigi. NEXUS auto-connects via XML-RPC on port 7362.
- The fldigi waterfall notice says "fldigi waterfall — connect fldigi to see live spectrum" when not connected.

Cause B: fldigi audio input not set to virtual device.

- In fldigi → Configure → Audio → Capture: set to BlackHole (macOS) or VB-Audio Cable (Windows).

Cause C: SDRConnect audio output not going to virtual device.

- In SDRConnect, set audio output to the same virtual device.

Cause D: fldigi frequency not matching NEXUS.

- Click ▶ **Start** to re-sync frequency.
- If fldigi has Hamlib/CAT configured pointing at port 4532, it should follow automatically.

fldigi waterfall is black / not showing

Cause: fldigi is not connected or `wf.get_data()` is returning empty.

- Confirm fldigi is running and responding (check its XML-RPC server is enabled: Configure → Misc → XML-RPC Server).
- Restart fldigi.

fldigi frequency doesn't match NEXUS frequency

Cause: If fldigi has Hamlib CAT configured pointing at a **different** port than 4532, it overrides NEXUS's frequency setting.

Fix:

- In fldigi → Configure → Rig Control: set to Hamlib NET rigctl, Host = localhost, Port = **4532** (NEXUS's rigctld port)
- **Or** disable fldigi's rig control entirely if you only want XML-RPC frequency sync

Why it happens: NEXUS sets fldigi's frequency via XML-RPC. If fldigi also polls a CAT interface, CAT overrides the XML-RPC setting every ~1 second. NEXUS's rigctld at port 4532 serves the correct dial frequency, so pointing fldigi at it keeps everything in sync.

RTTY decoder shows garbage / wrong characters

Check the Signal badge first (added July 2026). The RTTY Parameters panel now shows a **LOCKED** (green) or **NO SIGNAL** (red) badge with a live dB reading, next to the State line. If it reads NO SIGNAL, the decoder has correctly detected there's no real carrier at your current tuning/tones and any lingering text on screen is stale — retune or fix your tone settings rather than troubleshooting the text itself.

Background: earlier builds had no such check — RTTY has a continuous carrier (mark or space is always being transmitted) unlike CW's genuine on/off keying, so the decoder would compare mark-tone power to space-tone power and produce a bit either way, even on pure band noise. Two noise readings are "greater than" each other about half the time, which was enough to pass the start/stop framing check often enough to keep grinding out plausible-looking garbage indefinitely, with no visible sign anything was wrong. A live spectral squelch (checking a patch of the passband expected to be empty during a real

signal) now gates new character acquisition on an actual SNR threshold — this is what LOCKED/NO SIGNAL reflects.

If LOCKED but text is still unreadable: this points to the signal itself, not a setting. Check, in order:

- Baud rate matches the signal (try Auto-detect)
- Shift matches (170 Hz for amateur, 425/850 Hz for commercial)
- Charset is correct (ITA-2 for most amateur/commercial, MTK-2 for Russian)
- Mode is USB (not LSB)

If all of the above check out and the signal is still garbled, it may simply be a weak or marginal real-world HF signal — a low-but-locked SNR reading is consistent with a signal too weak to decode cleanly even though it's genuinely present, not a NEXUS fault. A longer raw recording (the **REC** button) analysed offline is the only way to distinguish this conclusively from a remaining decoder issue.

WEFAX / ACARS / POCSAG / AIS show nothing — check stream mode first

Cause: These four decoders are listed under the **FULL IQ** category in the DECODERS dropdown because they genuinely require a raw IQ stream. As of w0.2.3, IQ Lite and Compact modes are confirmed to never deliver raw IQ on the nRSP-ST (see the Waterfall & Spectrum section above) — so if SDRConnect's stream mode is anything other than **Full IQ**, none of these four will ever produce output, regardless of any other setting.

Fix: Confirm SDRConnect's stream mode is **Full IQ** (this is NEXUS's default for SDRplay devices as of w0.2.3). Then check each decoder's own specific requirements below.

WEFAX decoded image never appears, only a log line (fixed in w033)

Symptom: the WEFAX tab logs that lines are being decoded, but the image area itself stays blank — no picture ever builds, even after a full transmission completes.

Cause: the backend was already streaming each decoded line as a real PNG, but the frontend never actually read or drew it — it only logged a text line and never touched the image data or the image container element at all. Separately, two of the four call sites that feed the WEFAX decoder passed a keyword argument its `process_iq()` method didn't accept, throwing an exception that (like the FT8 bridge bug noted elsewhere in this guide) could silently break the call site it ran from.

Fix (w033): the frontend now decodes and draws the streamed image data as it arrives, and the mismatched call sites were corrected to match `process_iq()`'s actual signature.

SSTV/rtl_433: brand-new decoders in w033 — see the User Manual, Sections 6.16a/6.16b

These two decoders didn't exist before this fork; there's no prior-version behaviour to compare against. The one issue found and fixed during this build is below.

SSTV running causes the main spectrum/waterfall to progressively stutter and eventually freeze (fixed in w033)

Symptom: with the SSTV decoder started, the main spectrum and waterfall (not just the SSTV tab) get progressively slower over time — a few minutes in, updates visibly lag, and eventually the display can freeze solid. Stopping SSTV (or restarting NEXUS) resolves it immediately.

Cause: SSTV searches for its VIS leader tone by scanning its whole audio buffer so far; the buffer only ever gets trimmed once that search has actually locked onto (or conclusively rejected) a leader tone. If no valid SSTV signal is ever present — dead air, or a non-SSTV signal on the tuned frequency — that condition never fires, so the buffer grows without bound and each scan gets slower, at the cost of the same code path that computes the spectrum/waterfall data for every other tab.

Fix (w033): added a hard buffer-length cap (8 seconds) independent of whether a leader tone has ever been found, so the scan cost stays flat no matter how long SSTV runs without a valid signal. Verified with a 2-minute synthetic no-signal stress test: buffer size now plateaus instead of growing to several million samples, and per-cycle cost stays flat. Restart the NEXUS server for this fix to take effect if you were already running an older SSTV session — this is a backend/.py fix.

AIS Marine tab shows no vessels despite Full IQ (native decoder, added in w0.2.6)

Symptom: SDRConnect is in Full IQ mode, the AIS Start/Stop toggle is on, but the vessel table/map (AIS tab or Marine tab) stays empty.

Cause A: Not tuned to or near an AIS channel.

- The native decoder demodulates GMSK directly off the IQ stream, but only when the VFO is on or close to one of the two AIS data channels: **161.975 MHz (CH87B)** or **162.025 MHz (CH88B)**. Use the CH87B/CH88B Quick Tune buttons (AIS tab or Marine tab — see section 6.9 in the User Manual) rather than tuning manually.

Cause B: SDRConnect's stream mode is not actually Full IQ.

- The native AIS decoder has the exact same Full IQ requirement as WEFAX/ACARS/POCSAG — see "WEFAX / ACARS / POCSAG / AIS show nothing" above. Confirm the stream-mode indicator before assuming anything else is wrong.
- **Fixed in w031:** previously, starting AIS in Compact/IQ-Lite mode did nothing at all with no explanation — every other Full-IQ-only decoder already showed an on-screen warning toast in this situation, but the AIS start handler was the one exception that skipped it. As of w031, starting AIS without a genuine Full IQ stream now shows the same warning the other Full-IQ decoders do, so this is no longer a silent failure.

Cause C: AIS Start/Stop toggle is not actually on.

- Check the toggle in either the AIS tab or the Marine tab's AIS Vessels panel — it's the same control for both the native decoder and the UDP/AIS-catcher method, so it must be switched on either way.

Cause D (this is usually normal, not a bug): No vessels are actually in range.

- AIS is VHF and line-of-sight — typical real-world range is roughly 10–40 nautical miles depending on antenna height, terrain, and vessel traffic density. If you're inland, far from navigable water, or simply at a time/place with no nearby ship traffic, an empty vessel list is expected even with everything configured correctly. Try again near a coastline, harbour, or shipping lane, or at a time with more traffic.
- Only CRC-16-verified frames are ever shown (by design, to avoid false vessel entries), so a marginal/noisy signal that fails CRC will also simply show nothing rather than garbage data — this looks identical to "no vessels in range" from the UI, and is not distinguishable without checking the terminal log.

Cause E: Using the older AIS-catcher/UDP method instead of (or in addition to) the native decoder, and AIS-catcher itself isn't running.

- The native decoder is independent of AIS-catcher and works without it. But if you're specifically relying on the separate AIS-catcher program feeding NMEA to NEXUS over UDP port 10110 (e.g. to share one feed across multiple programs), confirm AIS-catcher is actually running and pointed at port 10110 on the machine running NEXUS. There is no AIS-catcher-specific setup section elsewhere in this guide at present — if you're setting this up for the first time, the native decoder above is the simpler path and requires no external program at all.

Cause F (fixed in w032): the decoder's bit-slicer had no tolerance for real-world carrier offset (SDR LO error, or the transmitter's own allowed ±500 Hz tolerance), so it could see genuine AIS bursts right on frequency yet produce zero CRC-verified frames. If you're still on w031 or earlier and everything else above checks out, updating to w032 or later resolves it.

AIS shows "Idle"/Start on a fresh tab even though it's actually running (fixed in w033)

Symptom: reload the browser tab (or open NEXUS in a second tab) while AIS is already running server-side — the AIS Idle/Active badge and Start/Stop button still show the old "Idle"/"Start" state until you click Start yourself, even though the decoder never actually stopped.

Cause: the frontend's `applyState()` function — which reads the full server state broadcast on connect/reconnect and updates every other decoder's badge/button — never read the `ais_active` field at all. Every other decoder's active-state was wired into this sync path; AIS was the one gap.

Fix (w033): `applyState()` now also syncs the AIS badge and Start/Stop button from `ais_active` on every state broadcast, so a fresh tab or reconnect immediately reflects whatever's actually running server-side, with no click needed.

Vessel table shows a decoder-source tag I don't recognise (w033 only, not a bug)

w033 runs a second AIS front end (ported from Dire Wolf's multi-slicer bit-sync approach) alongside the existing native decoder — both read the same Full IQ stream and merge into the same vessel table by MMSI, so a vessel decoded by either (or both) shows up once. See the User Manual, Section 6.9, for details. This is w033-only; w032 has only the single native decoder.

AIS decodes some vessels but fewer than expected, or mostly types with no name/position data

Symptom: the decoder is clearly working — some vessels appear — but the count seems low for a busy area, or the table fills mostly with MMSI-only rows (types 8/10/12, no name/position ever).

Cause A (improved in w032): the earlier decode chain tried to bit-sync continuously across background noise, and used a receive filter not shaped to AIS's GMSK modulation — both reduced how many marginal-signal bursts could be recovered. w032 adds burst/energy detection (only attempts to decode real transmissions, not noise) and a properly matched receive filter, roughly doubling the number of unique messages and vessels recovered on a real test capture versus the previous release.

- This narrows the gap to dedicated external tools (e.g. AIS-catcher) but doesn't fully close it — those also do coherent frequency tracking and soft-decision decoding, a larger DSP undertaking not included in this pass. If you need maximum decode yield specifically, running AIS-catcher

against the same IQ stream (or a REC-captured `.cf32` file — see section 3.4a1 in the User Manual) alongside NEXUS's native decoder remains the most complete option.

Cause B — not a bug: message types 8, 10, and 12 carry no name, callsign, speed, course, or position fields in the base AIS spec, regardless of decode quality. A harbour or shore-station-heavy area can genuinely have more of this traffic than ship position reports at any given moment — see "AIS vessel table fills with MMSI numbers…" below for the full explanation.

AIS vessel table fills with MMSI numbers but NAME/SOG/COG/STATUS stay blank

Cause A — aisstream.io isn't configured (affects NAME only). SOG/COG/STATUS never come from the online feed regardless — see Cause B below for those. (VesselAPI, an earlier online lookup source, was removed — aisstream.io is now the only online enrichment provider.)

- Check the **aisstream.io feed** field in the AIS tab: it reads either **not configured** or **configured (… XXXX)**. If not configured, paste a key (see "Online Vessel Name Lookup" in the User Manual, section 6.9a) and click Save — no restart needed.
- **If you set DARKSKY_AISSTREAM_KEY as an environment variable and launch NEXUS from IDLE (Run Module/F5)** rather than a Terminal: IDLE's process only inherits env vars from whatever shell launched IDLE itself, not automatically from `~/.zshrc`/`~/.bash_profile`, so a key that's genuinely exported there can still be invisible to NEXUS. Easiest fix: use the GUI field instead (writes to a local keyfile, works regardless of how NEXUS was launched).

Cause B — the vessel was only ever seen via a non-navigational message type. Not every decoded AIS message carries position/speed/course data. Message types 8 (Binary Broadcast — commonly sent by shore stations, harbour authorities, and AtoN buoys, not ships), 10, and 12 carry no nav or name fields at all in the base spec, so a vessel built entirely from one of those will show MMSI-only permanently, correctly, until it also happens to transmit a type 1/2/3/5/18/24 message.

- This is common near busy harbours, where shore infrastructure transmits alongside vessel traffic. It isn't a bug — those message types genuinely don't carry the fields the table shows.

Cause C — the MMSI itself may not be a real vessel identity. NEXUS rejects any decoded MMSI whose structure can't correspond to a real ITU-assigned identity (e.g. a Maritime Identification Digit outside the 201–775 range assigned to ships) before it's ever added to the table. If the vessel count drops noticeably after an update, this filter is why — those weren't resolvable vessels to begin with, whether from decode noise or a non-compliant transmitter.

ADS-B map shows no aircraft

Cause A: dump1090-fa or readsb is not running.

- macOS: `brew install readsb`, then run it with its web/JSON output enabled (default `--net`)
- NEXUS polls the JSON feed, trying `http://127.0.0.1:8080/data/aircraft.json`, then `http://127.0.0.1:8080/skyaware/data/aircraft.json`, then `http://127.0.0.1:8754/data/aircraft.json`, then falls back to the SBS text feed on port 30003.

Cause B: No RTL-SDR dongle attached (ADS-B requires a separate RTL-SDR tuned to 1090 MHz).

Cause C: Not started in NEXUS.

- Click ▶ **Start** in the ADS-B decoder panel.

WSPR decoder shows no spots

Cause A: wsprd is not installed.

- Install WSJT-X (wsprd is included) or install wsprd standalone.
- macOS: `brew install wsjtx`

Cause B: You need to wait for the top of a two-minute cycle (00:00, 02:00, 04:00 UTC etc.).

Cause C: Frequency is not a WSPR frequency.

- 20m: 14.0956 MHz, 40m: 7.0386 MHz, 30m: 10.1387 MHz

WSPR decodes fewer/weaker spots than expected (fixed in w031)

Symptom: WSPR Beacons decode cycles complete and occasionally produce spots, but noticeably fewer than the same signal conditions would yield in WSJT-X's own wsprd, especially on busier bands with several simultaneous transmitters.

Cause: The decimation step that brings the IQ stream down to wsprd's expected sample rate was a simple sample-drop (`audio = base[::dec].real`) with no anti-aliasing filter first. Dropping samples without filtering folds energy from above the new Nyquist frequency back down into the passband as aliased noise, degrading the effective SNR wsprd sees — exactly the kind of subtle quality loss that shows up as "fewer decodes on a marginal signal" rather than an outright failure.

Fix: Decimation now uses `scipy.signal.decimate` with an anti-aliasing FIR filter (zero-phase) when scipy is available, falling back to the old plain-drop method only if scipy isn't installed. `pip install scipy` if you haven't already — see Requirements in the Quick Start guide.

WSPR via RTL-SDR: decodes worse than SDRConnect/SDRplay path (fixed in w031)

Symptom: WSPR decoding on the RTL-SDR (`--rtl`) path performs noticeably worse than the same antenna/signal via SDRConnect.

Cause: The RTL-SDR call site was passing the *display* sample rate (`state['sample_rate']`, the UI's selected value) to the WSPR decoder instead of the actual post-decimation sample rate of the audio being handed to it, so the decoder's internal resampling math was working from the wrong reference rate.

Fix: The RTL-SDR WSPR call now passes the real decimated sample rate (`_rtl_dec_sr`) instead of the display value. No user action needed beyond updating to w031.

FT8: enabling it used to disconnect/reconnect SDRConnect repeatedly (fixed in w030)

Symptom: turning on FT8 (WSJT-X-bridge mode) caused the SDRConnect connection to drop and reconnect every few seconds — spectrum/waterfall/audio would glitch or freeze briefly, over and over, for as long as FT8 stayed enabled. The log (terminal NEXUS was launched from) showed repeating `SDRConnect bridge error … — retrying in 5s` messages the whole time.

Cause: a backend bug present since this decoder class was written (not something w030 introduced — inherited silently through every build up to and including w029). The FT8 decoder object only ever tailed WSJT-X's `ALL.TXT` file for spots; it had no code path to handle live audio. But several places in the audio-processing pipeline called methods on it that didn't exist, and did so on every single audio packet

once FT8 was turned on — each call threw an error that silently tore down and reconnected the entire SDRConnect link, then did it again on the next packet.

Fix: those calls have been removed in w030. Enabling FT8 (either source) no longer touches the SDRConnect connection at all beyond what every other decoder already does. If you're on an older build (w026–w029) and see this, update to w030 or later.

FT8 INTERNAL source: Decode is on, but nothing ever decodes (new in w030)

First, confirm the basics: FT8 SOURCE is actually set to INTERNAL (not WSJT-X), you're tuned to an FT8 or FT4 frequency in USB mode, and you've clicked ▶ **Start** in the tab (this replaced an earlier single "Decode On/Off" toggle — the control is now a Start/Stop pair, matching every other decoder in NEXUS).

Cause A — no audio is reaching the browser.

The INTERNAL decoder does not use your microphone — there is no audio device to select. Its audio comes from the same WebSocket connection as everything else, streamed from the backend only while INTERNAL is selected and the decoder is running. If SDRConnect/RTL-SDR itself isn't actually streaming (check the main spectrum/waterfall is live), there's nothing for the decoder to receive either. Confirm the connection status shows SDRConnect (green) or RTL-SDR, not OFFLINE.

Cause B — switched source mid-session without restarting.

Toggling FT8 SOURCE doesn't automatically start decoding — it only decides which panel (and which audio path) is active. After switching to INTERNAL, click ▶ **Start** if it isn't already running.

Cause C — first decode cycle hasn't completed yet.

FT8 decodes in 15-second windows (FT4: 7.5 seconds) aligned to the top of each cycle, same as WSJT-X. The status line shows "Buffering…" until the first full window has been collected — this can take up to one cycle length after pressing Start, which is normal, not a fault.

Cause D — weak/no signals on the tuned frequency right now.

Confirm there's genuine FT8/FT4 activity on the band and frequency you're tuned to — try a busier band (20m/14.074 MHz is usually reliable during daylight) before assuming the decoder itself is broken.

The propagation map doesn't render at all / shows an error: the map loads tiles from an internet CDN — which provider depends on the style dropdown at the top of the map column (CARTO by default, or OpenStreetMap/Esri if selected). If the machine running NEXUS has no general internet access, the map can't load — this doesn't affect decoding itself, only the map visualization.

My spots vanished when I switched bands: fixed July 2026 — manual band-switching used to wipe the map and decode list on every click. It no longer does (see the User Manual's Propagation Map section); if you're still seeing this, hard-reload the browser.

FT8 INTERNAL: decoder runs every cycle, tones are clearly visible, but it always decodes 0 (Full IQ mode, fixed in w033 — see the 2026-07-20 update below if you're on a directly-connected SDRplay device)

Symptom: FT8 SOURCE is INTERNAL, Start is pressed, the status line correctly cycles through Buffering/Decoding/Done every 15 seconds, and the mini-scope clearly shows strong FT8 tones — but the decode count stays at 0 indefinitely, even on a busy band.

Cause 1 — sample-rate mismatch (2026-07-19, first fix attempt). Full IQ's audio-tap rate is variable (chosen to land close to 48000 Hz without dropping below it — at 250 kSPS it lands on 50000 Hz instead), but the browser-side decoder hardcoded a fixed 48000→12000 Hz conversion. A ~4% rate error is enough to shift FT8's whole tone passband out of sync range. Real, but not the whole story — see Cause 2.

Cause 2 — wrong signal entirely (2026-07-19, actual blocker for this round). Full IQ's FT8 audio broadcast was sending the real part of the wideband IQ centred on the receiver's hardware local oscillator, not on the tuned VFO — never a demodulated single-channel audio signal in the first place, so no sample-rate fix could have corrected it. On networked nRSP-ST units, SDRConnect was found to also send its own correctly demodulated, VFO-centred audio continuously regardless of display mode (the same stream Compact mode uses), so switching to Compact worked around it there.

First fix (2026-07-19) — later found incomplete: removed the Full IQ branch's own audio broadcast entirely, reasoning that the always-present Compact-mode broadcast would supply the FT8 buffer instead, in every stream mode. This only holds for networked nRSP-ST units.

Cause 3 — RSPdx/direct-USB devices have no Compact-mode audio at all (2026-07-20, second report: "ive just started ft8 internal, and it not showing decodes again like last time"). A directly-connected RSPdx (`device_type: "RSPdx"`, not `"nRSP-ST"`) never receives a Compact-mode audio broadcast from SDRConnect in the first place — Full IQ is its only mode (see the "Full IQ (only mode)" status badge). Removing the Full IQ branch's own broadcast in the first fix left these devices with zero path to the FT8 decoder at all. Confirmed live via a browser-side packet counter: zero audio frames arrived over a 15+ second FT8-enabled window with a strong on-screen signal.

Final fix (2026-07-20): restored the Full IQ branch's own audio broadcast, this time correcting the actual defect the original version had instead of removing it — the signal is now mixed down by the VFO-to-hardware-centre offset (shifting the tuned frequency to 0 Hz) before the real part is taken, so it's genuinely VFO-centred audio, not raw LO-centred IQ. Both device types now get a working FT8 feed in Full IQ: nRSP-ST via the (unaffected) Compact-mode broadcast, RSPdx/direct-USB via this corrected Full IQ broadcast.

If you're still seeing 0 decodes in Full IQ mode after updating, confirm you've fully restarted the NEXUS server — this is a backend/`.py` fix, not something a browser refresh alone picks up.

Status bar shows "Full IQ (only mode)" and I don't have a direct USB connection — is that a bug?

Not a bug — the badge was misworded, now fixed (2026-07-20). It previously read "Full IQ (USB direct)", which implied NEXUS had verified a literal local USB cable between this machine and the radio. It never actually checked that — it only checks whether SDRConnect reports a `device_type` other than `"nRSP-ST"`, which is true for any RSPdx-class device whether it's plugged into this machine or reached over a network/SSH to a remote one.

The badge still shows for the same reason as before: this device type has no selectable Compact/IQ-Lite mode in SDRConnect (only networked nRSP-ST units get that mode selector), so it's always genuinely Full IQ once connected. The relabeled "Full IQ (only mode)" states that fact without the unverified USB claim.

RTTY auto-detect confidently shows "85% / GOOD / Signal locked" but the decoded text is scrambled (fixed 2026-07-20)

Symptom: with Auto-detect baud & shift enabled, the RTTY panel showed a steady "85% confidence, Signal locked, GOOD" reading and had locked onto a mark/space pair — but the decoded text was garbage, and the live decoder's own separate squelch simultaneously reported NO SIGNAL on the same audio.

Cause, two bugs stacked: the auto-detect algorithm only ever checked *relative* fit (is the second tone peak at least 20% of the first, do the bit timings statistically resemble a standard baud rate) — never whether the detected tones carried any real power above the noise floor, so a filter-edge artifact could pass both checks. Separately, the confidence readout was never actually computed by the backend at all — the frontend silently defaulted to a hardcoded 85% on every single reply, real detection or not, regardless of the underlying signal quality.

Fix: auto-detect now gates on absolute signal-to-noise ratio using the same technique the live decoder's own squelch already uses, and reports a genuine confidence score. A weak or noise-only capture is now correctly rejected (or shown at low confidence) instead of being reported as an 85% "GOOD" lock.

RTTY tone scope waterfall/spectrum visibly "jumps" during live decoding (fixed 2026-07-20)

Symptom: with a decoder running (and especially with Auto-detect on), the Tone Scope's spectrum trace/curve — and sometimes the M/S mark/space markers themselves — visibly jumped or reshuffled every second or two, rather than holding a steady picture of the signal.

Cause 1 — auto-detect retuned on every capture, regardless of confidence. Every ~1-second auto-detect capture was applied immediately (moving the mark/space markers and re-tuning the live decoder), even when that particular capture's confidence was low. On a marginal signal, successive captures can each land on a slightly different candidate tone pair, so the markers kept hopping between them.

Cause 2 — two different data sources competing for the same display. Independently of auto-detect, the Tone Scope was also receiving spectrum data from two different backend sources at once — one correctly centred on the live tone and one that wasn't, each at a different resolution — because a networked SDRplay unit (nRSP-ST) streams a continuous audio feed regardless of which display mode (Compact/IQ-Lite/Full IQ) is selected, and both the always-on feed and the display-mode-specific feed were being sent to the browser.

Fix: auto-detect now only retunes the live decoder once its confidence clears a reasonable threshold — a weak capture updates the on-screen quality label but no longer moves the markers or the live tuning. The redundant second data source has also been suppressed for networked (nRSP-ST) units, so the Tone Scope now has exactly one feed instead of two disagreeing ones. If you still see jumping after updating, confirm you've fully restarted the NEXUS server — this is a backend/`.py` fix, not something a browser refresh alone picks up.

Reporting / Logbook Issues (w033 only)

Reporting tab / Logbook doesn't exist in my build

This is expected on w031 and w032. The REPORTING tab (Logbook for SWL reception reports, FT8 decodes, and manual entries) was added in w033 only — see the User Manual, Section 9a. It has no equivalent in earlier builds.

Populate from FT8 button says "No FT8 decodes captured yet this session"

Cause: Populate from FT8 only works from decodes captured during the *current* browser session — it hooks the same `appendFT8Decode()` function the FT8 tab uses to render each decode as it arrives, and buffers up to the last 500. It does not read from any saved FT8 log file, and a page reload clears the buffer along with everything else in memory.

Fix: start the FT8 INTERNAL decoder (see Section 6.6) and let at least one decode cycle complete, then click **Populate from FT8** again. If FT8 itself isn't decoding, see the FT8 troubleshooting entries above first.

Populate from FT8 reports "0 new entries" even though FT8 is actively decoding

This is the expected duplicate-prevention behaviour, not a bug. Every FT8 decode is deduped by a signature of its timestamp, decoded message text, and frequency — clicking Populate repeatedly (or reopening the Reporting tab and clicking it again) only adds decodes that haven't already been logged. If you expect new decodes and see none added, confirm the FT8 tab's own decode count is actually increasing, not just that the decoder is running.

A logged FT8 entry's notes field is missing distance/bearing/country

Cause: distance, bearing, and country are only computed when the decoded FT8 message contains a 4-character (or longer) grid locator — many FT8 exchanges (later stages of a QSO, or messages that only carry an RST/73) don't include a grid at all, so there's nothing to convert to a position. Country lookup additionally depends on the callsign parsing correctly against NEXUS's prefix table.

Not a bug: the entry is still logged with whatever fields the message actually gave you — callsign, frequency, mode, SNR, dt, and the raw message text are always present regardless of whether a grid was decoded.

Deleted or edited a log entry but it reappeared / edit didn't stick

Cause: the Logbook is a server-authoritative list, same as Bookmarks — the browser never edits its own copy directly. Add/Edit/Delete all send a command to the NEXUS server, which updates `darksky_logbook.json` and then re-broadcasts the full list back to every connected browser tab. If the WebSocket connection drops right as you save or delete, the change may not have reached the server at all.

Fix: check the connection status indicator (top bar) — if it shows disconnected, reconnect and retry the edit/delete. If entries still don't persist across a full NEXUS restart, confirm the application folder is writable (same requirement as Bookmarks and Frequency Lists).

FT8/WSPR: frequency control works (NEXUS can tune WSJT-X) but no decodes ever appear

(This section is about the WSJT-X source. If you're using the new INTERNAL native decoder instead, see the previous entry above.)

This is the most common FT8 report — NEXUS and WSJT-X use two completely different, one-way channels, and it's easy to have one working without the other:

- **NEXUS → WSJT-X (frequency control):** NEXUS sends UDP Heartbeat/DialFrequency packets to 127.0.0.1:2237. This is outbound only — it tells WSJT-X what to tune to, but WSJT-X never sends anything back over this port.
- **WSJT-X → NEXUS (decodes):** NEXUS does **not** listen on a UDP/network port for decodes. Instead it tails WSJT-X's own ALL.TXT log file on disk, polling it for new lines every 2 seconds. If you can tune WSJT-X from NEXUS but see no spots, the file-tailing side is the thing to check — the UDP control link being fine tells you nothing about whether ALL.TXT is being found.

Cause A — WSJT-X is on a different computer than NEXUS.

Because the decode path reads a local file, this **only works when WSJT-X runs on the same machine as NEXUS**. If WSJT-X is on another PC (e.g. you're running NEXUS remotely, or WSJT-X on a separate logging machine), NEXUS has no way to read its ALL.TXT — there is no remote/network fallback for this. Run WSJT-X locally alongside NEXUS, on the same computer that's connected to the SDR.

Cause B — `ALL.TXT` doesn't exist yet, or isn't where NEXUS expects it.

NEXUS searches, in order:

```
macOS:   ~/Library/Application Support/WSJT-X/ALL.TXT
         ~/Documents/WSJT-X/ALL.TXT
Linux:   ~/.local/share/WSJT-X/ALL.TXT
Windows: %APPDATA%\WSJT-X\ALL.TXT
         (current working directory)/ALL.TXT
```

WSJT-X only creates this file once it has actually decoded at least one message, and it always logs to the standard per-OS location above (not configurable). If you've just started WSJT-X and it hasn't decoded anything yet, wait for at least one real decode cycle, or check the terminal NEXUS was launched from for FT8/WSPR: WSJT-X ALL.TXT not found — spots will appear once WSJT-X runs (logged once at decoder start, and again the moment the file is found). NEXUS re-checks for the file every poll, so once ALL.TXT appears, spots should start showing within ~2 seconds with no restart needed.

Cause C — the FT8/WSPR decoder panel was never started in NEXUS.

File-tailing only begins once you click ▶ **Start** in the FT8/WSPR decoder tab. Tuning/frequency control works independently of this — so it's possible to be actively retuning WSJT-X while the NEXUS-side decode reader has never actually been switched on.

Cause D — WSJT-X mode doesn't match what you're tuned to, or WSJT-X isn't decoding at all.

Confirm WSJT-X's own waterfall/decode log shows activity for your current frequency and mode before assuming NEXUS is at fault — NEXUS only displays what WSJT-X has already decoded and logged.

PSK Reporter — my spots never show up on pskreporter.info (new in w031)

Cause A: Callsign not set, or reporting not toggled on.

- Reporting requires a valid callsign, set in the HF Utility tab's location & callsign bar, with the PSK Reporter checkbox turned on there too. Your grid locator is derived automatically from the location already set in the same bar — there's no separate locator field to fill in. Spots are silently not sent if the callsign is missing.

Cause B: Nothing has actually decoded yet.

- PSK Reporter only receives spots from decodes NEXUS itself makes (FT8/FT4, WSPR, or CW Skimmer) — it doesn't report anything on its own. Confirm the relevant decoder is running and actually producing decodes in its own panel first.

Cause C: Outbound UDP to port 4739 is blocked.

- Spots are sent via UDP directly to report.pskreporter.info:4739. If you're behind a restrictive firewall or a network that blocks outbound UDP, the packets never arrive — check your firewall/router isn't blocking that port.

Cause D: Rate limiting / normal delay.

- Uploads are batched rather than sent per-decode, so there can be a short delay (up to a few minutes) between a decode appearing in NEXUS and the same spot appearing on pskreporter.info. Give it a few minutes before assuming it isn't working.

Cause E: CW Skimmer spot specifically didn't have an extractable callsign.

- For CW, NEXUS uses the same CQ/DE heuristic parsing as the Skimmer's own callsign column. If a decoded message doesn't clearly contain a recognisable callsign pattern, it's skipped rather than guessed at — this is expected for partial or garbled copy, not a bug.

Cause F (fixed later in w031; check you're on the current build): you were using FT8 INTERNAL, not the WSJT-X bridge.

- User-reported: 828 decodes / 118 callsigns / 20 countries showing in NEXUS's own FT8 INTERNAL panel, nothing under their callsign on pskreporter.info. FT8 INTERNAL decodes entirely inside the browser tab and, for a portion of the w031 cycle, never sent a single decode to the Python backend — the only process that can actually open a UDP socket to PSK Reporter. This was fixed the same day it was reported; if you still see this, hard-reload the browser (Cmd+Shift+R / Ctrl+Shift+R) and restart the Python server to make sure you're on the fixed build, then re-check the WSJT-X-bridge-vs-INTERNAL note in Section 6.6a of the User Manual.

HFDL / VDL2 shows nothing

Cause: dumphfdl / dumpvdl2 is not running or not outputting JSON to the expected port.

- dumphfdl: must output JSON to localhost port 5556
- dumpvdl2: must output JSON to localhost port 5557
- Click ▶ **Start** in the respective decoder panel — NEXUS will attempt to launch the tool automatically if it is in your PATH.

Multimon-ng / POCSAG not decoding

Cause A: multimon-ng is not installed.

- macOS: `brew install multimon-ng`

Cause B: Audio not routed to virtual device.

Cause C: Wrong modes selected in the mode selector panel.

Cause D: SDRConnect stream mode is not Full IQ — see "WEFAX / ACARS / POCSAG / AIS show nothing" above.

Status bar shows "not running" or looks frozen (fixed in w0.2.6): Earlier builds only sent the AUDIO/PID/uptime heartbeat from the RTL-SDR-direct code path, so on every SDRConnect stream mode (PCM, IQ-Lite, Full IQ, Compact) the status bar would freeze at its startup values — uptime stuck at 0, AUDIO meter never moving — even while multimon-ng was genuinely decoding in the background. As of w0.2.6 the heartbeat is sent from inside the shared decode path itself, so it now updates roughly every 2 seconds on all stream modes. If the AUDIO meter (with peak-hold tick and scrolling sliver chart), per-mode pill counters, and PID/uptime readout are all moving, multimon is working — a quiet message list with an active status bar usually just means no traffic on the modes/frequency you're currently on, not a decode failure.

No quick-tune button for your mode: The Multimon tab's Quick Tune panel only has buttons for modes with a genuinely universal frequency — POCSAG/FLEX pager blocks (UK/EU 153.025 MHz; US 929.6125 MHz and 152.480 MHz), APRS, and NOAA Weather Radio (162.400/162.550 MHz, used as a stand-in tune point for EAS, which has no dedicated frequency of its own). DTMF and the selcall variants (ZVEI1/2/3, DZVEI, PZVEI, EEA, EIA, CCIR) have no universal frequency — they're country- or agency-specific — so you'll need to tune manually for those.

Numbers-Station / HF-Intel (Rivet) decoder shows nothing

Cause A: Wrong mode selected — DSC and Baudot use different tone schemes.

- DSC (CCIR 493-4): 100 Bd, 170 Hz shift, ~1700 Hz tone centre.
- Baudot (ITA2 RTTY): 50 Bd, 170 Hz shift.

Cause B: Not tuned correctly.

- Use one of the DSC Quick Tune buttons in the panel (2187.5 / 4207.5 / 6312 / 8414.5 / 12577 / 16804.5 kHz) — these set USB, 300 Hz BW automatically.

Cause C: Decoder not started.

- Click ▶ **Start** in the panel header.

Note: This decoder is a clean-room reimplementation of the public ITA2 (Baudot) and ITU-R M.493-9 (DSC) protocol specifications — it shares no code with the third-party Rivet project, which has no published license. No external binary or install is required; it runs natively in NEXUS.

FreeDV decoder shows "freedv_rx not found"

Cause: The `freedv_rx` demo binary from the codec2 project is not on your PATH. It is not included in any standard package manager — it has to be built from source.

Fix (macOS/Linux):

```
git clone https://github.com/drowe67/codec2
cd codec2
mkdir build_linux && cd build_linux   # name doesn't matter, used on macOS too
cmake ..
make
```

The binary will appear at `codec2/build_linux/src/freedv_rx`. NEXUS looks for it on PATH and in several common build locations (`~/codec2/build_linux/src/freedv_rx`, `/usr/local/bin/freedv_rx`, `/opt/homebrew/bin/freedv_rx`). If you built it somewhere else, either copy it to one of those paths, add it to your PATH, or move/symlink the binary so it appears at `~/codec2/build_linux/src/freedv_rx`.

Fix (Windows): codec2 has no official Windows build; you'll need to build it yourself (e.g. via MSYS2/MinGW or WSL) and place `freedv_rx.exe` at one of the paths NEXUS checks (see the FreeDV tab tooltip), or add it to PATH.

FreeDV decoder runs but never syncs / no audio

Cause A: Wrong mode for the signal. Try a different mode in the dropdown — 700D is the most robust for weak/noisy HF signals, 1600 and 2400A/B expect a cleaner, stronger signal.

Cause B: Not tuned precisely enough, or BW too narrow/wide. FreeDV HF modes are USB, narrow digital-voice channels (~1.5–2 kHz) — NEXUS sets BW to 2000 Hz automatically when you press Start, but you may need to nudge the VFO by a few tens of Hz to find sync.

Cause C: Browser blocked audio autoplay. Most browsers require a user gesture (a click) before playing audio — clicking **▶ Start** counts as that gesture, so if you reload the page after the decoder is already running, audio may not resume until you click Start again or interact with the page.

Cause D: Volume slider is at 0, or Mute is on — check both controls in the FreeDV tab.

Freq Lists — "On Now" filter lights up but all entries still show (fixed in w0.2.6)

Symptom: Pressing the **On Now** button in the Freq Lists tab highlights it as active, but the list of entries doesn't change — all entries remain visible.

Cause: The On Now filter reads the `time` field stored for each entry at import time. If `time` is empty for every entry, the filter sees no time data and shows everything. This happens most often with **SwSKEDS format**, which uses 'On' and 'Off' column headers rather than the 'time' or 'utc' headers the parser originally recognised. Earlier builds left `e.time = ''` for all SwSKEDS entries as a result.

Fix (w0.2.6): The parser now detects 'On'/'Off' column headers and combines them into a `HHMM-HHMM` time string per entry. However, this only applies to **newly imported** lists — existing entries already saved in localStorage have empty time fields that can't be retroactively filled. **Delete and re-import your SwSKEDS list** after updating to w0.2.6 or later, and the On Now filter will then work correctly.

If you're not using SwSKEDS format, check that your source file has a column named 'time', 'utc', or similar — those are the other supported time-column headers.

Freq Lists — Language dropdown shows time-like values (e.g. "0700-1900") (fixed in w0.2.6)

Symptom: The **All Languages** dropdown in the Freq Lists filter bar contains entries that look like time ranges (e.g. "0700-1900") rather than language names.

Cause: Column mis-detection during import caused time-range strings to be stored in the `language` field for some entries, or the dropdown's build function was aggregating values from the wrong field.

Fix (w0.2.6): The language dropdown builder now excludes any value matching the `HHMM-HHMM` time-string pattern. Re-import your list if the dropdown is still showing stale values from before the fix, or simply re-open the Freq Lists tab to rebuild the dropdown from the current data.

Bookmark popup closes by itself when typing, tabbing, or pressing Enter (fixed in w0.2.6)

Symptom: The Bookmark popup (opened via the 🔖 Bookmark button on the top command bar) closes unexpectedly as soon as typing starts in the Name, Notes, or Mode fields — or pressing Enter in a field dismisses the popup before saving.

Cause: The global `window.addEventListener('keydown', ...)` handler fired for every keystroke regardless of which element had focus. Single-letter shortcut keys assigned to UI toggles (e.g. `c` = toggle Cinematic mode, `r` = open Radar, `d` = open DSP) were triggered by letters typed in the popup, causing sudden UI changes that made the popup appear to vanish. Pressing Enter was interpreted by the browser as activating the first focused button (Cancel or Save Bookmark), dismissing the popup before the user finished typing.

Fix (w0.2.6): The keydown handler now checks `document.activeElement` at the top and returns immediately if focus is in an INPUT, TEXTAREA, SELECT, or contentEditable element. Single-letter shortcuts are therefore suppressed while any input is active. Additionally: **Enter** in any Bookmark popup field now saves the bookmark; **Escape** cancels and closes the popup; `stopPropagation()` is added to the popup box to prevent any accidental event bubbling to the overlay backdrop. Hard-reload the browser (Cmd+Shift+R / Ctrl+Shift+R) if the popup is still misbehaving after updating — this is a frontend-only fix.

Bookmark popup still closes while typing/editing text — different from the w0.2.6 fix above (fixed in w033)

Symptom: typing right after clicking the Bookmark button opens some other panel or triggers a keyboard shortcut instead of entering text — the field never receives the keystrokes at all, rather than the popup dismissing outright.

Cause: the popup deferred focusing the Name field with `setTimeout(fn, 0)` after making itself visible, leaving a brief real window where the previously-focused Bookmark button was still `document.activeElement`. Any keystrokes typed in that window fell through to the global single-key shortcut handler (the same handler the w0.2.6 fix above already guards for input focus — but this gap happened *before* focus ever moved to the input, so that guard didn't catch it).

Fix (w033): the Name field is now focused synchronously, the instant the popup becomes visible, closing the gap entirely.

Bookmark popup closes when you select/drag-select text inside it (fixed in w033)

Symptom: editing or typing in the bookmark fields works fine, and Tab-navigating between fields works fine — but selecting text with the mouse (a drag-select, or a triple-click/"select all") inside any of the three fields closes the popup.

Cause: the popup's backdrop closes on any click whose target is the backdrop itself; the popup box's own click handler normally prevents this by stopping the event from bubbling out — but only for clicks that originate inside the box. A text-selection drag that starts inside the (fairly narrow) box but overshoots past its edge before the mouse is released ends with the resulting click's target being the backdrop directly, bypassing the box's own protection entirely.

Fix (w033): the backdrop now only closes the popup if *both* the initial mousedown and the resulting click land on the backdrop itself, not just the click — a selection that starts inside the box no longer counts, however far the drag travels.

SSH Launcher Issues

SSH connection fails immediately

Check:

- Host IP and port are correct
- Username and password are correct
- SSH is enabled on the remote machine
- Firewall allows port 22 (or custom SSH port)
- Try connecting with a terminal first: `ssh user@host`

SDRConnect starts on remote but NEXUS can't connect

Cause: SDRConnect WebSocket on the remote is not accessible locally. You need SSH port forwarding.

Fix: Add this to your SSH connection (in the NEXUS SSH config): ensure you have forwarded port 5454 from the remote to localhost. NEXUS then connects to `localhost:5454`.

"SIGSEGV in swig_bindings" after stop/restart

Cause: SDRConnect was restarted before the device released.

Fix: Increase the **Device release wait** in SSH config (default 5 seconds, try 8–10 seconds for RSPdx). Do not restart SDRConnect within 10 seconds of stopping it.

Performance Issues

App takes 5-15 seconds to launch (packaged .exe / .app builds)

Cause: NEXUS's own startup code is lean — it doesn't do any blocking file or network reads before the server binds and the browser opens. The delay reported by beta testers on Windows and macOS is

consistent with the OS (or antivirus, e.g. Windows Defender/SmartScreen) scanning the unsigned packaged build and its bundled libraries (scipy, numpy, sounddevice, paramiko — roughly 100MB) the first time it runs, or on every run if it's quarantined/re-scanned. This is a one-folder PyInstaller build, not a one-file build, so there's no per-launch archive-extraction overhead either — the slowdown is happening before NEXUS's own code even starts running.

Fix: There's no in-app fix for this — it's an artifact of the build being unsigned, not application code. Code-signing the release build (Apple Developer ID notarization on macOS, an EV code-signing certificate on Windows) is the actual fix, planned for a future release process change. In the meantime, if your antivirus supports it, adding an exclusion for the NEXUS folder can reduce the delay on subsequent launches (the first scan is unavoidable either way). This is separate from the one-time Gatekeeper/SmartScreen warning dialogs — see the macOS Specific and Windows Specific sections below for those.

Waterfall is choppy / slow

Cause A: Browser tab is not in focus (Chrome throttles background tabs).

- Keep the NEXUS tab in the foreground.

Cause B: High CPU load from decoder processing.

- Disable decoders you are not using (click Stop).
- Install scipy for optimised filter processing: `pip install scipy`

Cause C: Large FFT size.

- Reduce FFT size if it is set very high.

Audio is choppy (RTL-SDR mode)

Cause A: rtl_tcp sample rate too high for your CPU.

- Use `-s 1024000` with rtl_tcp instead of higher rates.

Cause B: scipy not installed — fallback demodulator is less efficient.

- `pip install scipy`

Cause C: sounddevice not installed.

- `pip install sounddevice`

Python / Startup Errors

This whole section is specific to running NEXUS from Python source. If you installed the .dmg or Windows installer, all dependencies are already bundled — none of these errors should occur, and if something does go wrong it isn't this.

`ModuleNotFoundError: No module named 'websockets'`

```
pip install websockets
```

`ModuleNotFoundError: No module named 'numpy'`

```
pip install numpy
```

`OSError: [Errno 48] Address already in use` (macOS/Linux) or `OSError: [WinError 10048]` (Windows) — port 8888 or 8889

Cause: A previous NEXUS instance didn't shut down cleanly (a crash, a Task Manager "End Task", or launching a second copy before the first finished starting), or another app is using the port.

Fix — usually automatic (added w031): NEXUS now self-heals on startup. It checks whether the port is already held, and if the process holding it looks like a stale NEXUS/Python instance, it kills it and retries the bind automatically — on macOS/Linux via `lsof`/`ps`/`kill`, on Windows via `netstat`/`tasklist`/`taskkill`. In most cases the app just starts normally with no user action needed.

If it still fails (the port is held by something that doesn't look like NEXUS — the self-heal deliberately won't touch an unrelated process):

- macOS/Linux: `pkill -f w033_NEXUS.py`, or quit the stale "DARKSKY NEXUS" process in Activity Monitor if running as a packaged app
- Windows: End the stale "DARKSKY NEXUS" process in Task Manager, or run `netstat -ano | findstr :8889` to find the PID and `taskkill /F /PID <pid>`
- Or, if running from source, change `HTTP_PORT` / `WS_PORT` in the .py and restart

`Permission denied` when running on Linux

Some systems require elevated privileges for ports below 1024. NEXUS uses ports above 1024 by default so this should not occur. If it does, run with `sudo` or use `setcap`.

`SyntaxError` on startup

Cause: Python version is too old (2.x or 3.7/3.8).

Fix: Upgrade to Python 3.9+. On macOS: `brew install python3`.

macOS Specific

"...app cannot be opened because it is from an unidentified developer"

Cause: the .dmg build isn't code-signed or notarized. This warning is expected on first launch of any unsigned app, not a sign of a corrupted download.

Right-click (Control-click) **DARKSKY NEXUS w033 macOS** in Applications → **Open** → **Open** again in the confirmation dialog (to bypass Gatekeeper the first time). Subsequently it opens normally, including via a normal double-click.

Chrome not found / default browser opens instead

NEXUS looks for Chrome at /Applications/Google Chrome.app. Install Chrome from google.com/chrome or the URL will open in Safari instead.

BlackHole not appearing in SDRConnect audio output

- Install BlackHole 2ch from https://existential.audio/blackhole
- Restart SDRConnect after installation
- BlackHole appears as "BlackHole 2ch" in audio device lists

Windows Specific

"Windows protected your PC" (SmartScreen) when running the installer or launching NEXUS

Cause: the installer and app aren't code-signed with an EV certificate, so SmartScreen flags them as unrecognised on first run. This is expected for this build, not a sign of a corrupted download.

Click **More info**, then **Run anyway**. This is only needed once per download — subsequent launches from the Start Menu shortcut run normally.

Browser doesn't open automatically

NEXUS looks for Chrome at the standard install paths. If Chrome is installed in a custom location, open http://127.0.0.1:8888 manually.

`RuntimeError: Event loop is closed`

Cause: Windows requires a specific event loop policy for asyncio with sockets.

Fix: This is handled automatically in NEXUS (WindowsSelectorEventLoopPolicy). If you see this, ensure you are running from __main__ and not calling asyncio.run() yourself.

VB-Audio Cable not appearing in fldigi

- Install VB-Audio Cable from https://vb-audio.com/Cable
- Restart fldigi and SDRConnect after installation
- Select "CABLE Input" in SDRConnect audio output and "CABLE Output" in fldigi capture

Related Tools Evaluated But Not Integrated

SigDigger

SigDigger (https://github.com/BatchDrake/SigDigger) is a free, open-source blind signal analysis GUI (spectrum/waterfall inspection, Doppler analysis, burst detection, generic ASK/FSK/PSK demodulation). It was evaluated for inclusion in NEXUS and **not integrated**, for two reasons:

6. **Interface mismatch.** SigDigger is a standalone interactive Qt application with no headless/batch mode and no CLI or library entry point designed for being driven by another program — unlike multimon-ng, fldigi, or codec2's `freedv_rx`, which all expose a stdin/stdout or XML-RPC contract NEXUS can wrap. SigDigger's analysis tools are delivered through GUI workflows, not a pipeable decode pipeline.
7. **Licensing.** SigDigger and its `suscan` library are LGPL-3.0 (fine to link/wrap), but the underlying `sigutils` DSP library it depends on is GPL-3.0. Wrapping a standalone SigDigger *binary* as a subprocess would be unaffected by this, but lifting any of its actual blind-analysis algorithms into NEXUS's own code would require NEXUS to be GPL-compatible at that point.

If similar blind-signal-analysis features (Doppler estimation, burst/transition detection) are wanted in NEXUS in future, the realistic path is a clean-room reimplementation from public DSP literature, not a SigDigger wrapper or code import — similar to how the Rivet-inspired numbers-station decoder was built from public protocol specs rather than from Rivet's source.

In the meantime, SigDigger remains a good complementary tool to run alongside NEXUS for deep interactive signal investigation — just not something NEXUS launches or controls itself.

Getting Help

If an issue is not covered here:

8. If running from source, check the terminal output — NEXUS logs detailed info/debug messages there. (If running the installed app, launch it from Terminal/Command Prompt instead of the Applications/Start Menu icon to see the same log output.)
9. (source only) Enable debug logging by setting `logging.basicConfig(level=logging.DEBUG)` at the top of the .py
10. Note the exact error message and which step triggers it